



DEPARTMENT OF COMPUTER ENGINEERING (DCE)
SOC DESIGN
MMH: SOCD334264

Group: Anh em vi mạch K24

Name:

1. Lê Quốc Khánh - 24161276
2. Đặng Quang Được - 24161217
3. Lê Minh Hải - 24161225
4. Nguyễn Mai Trung Dũng - 24161206
5. Nguyễn Hữu Đức - 24161221
6. Lã Phan Ân Kỳ - 24161294

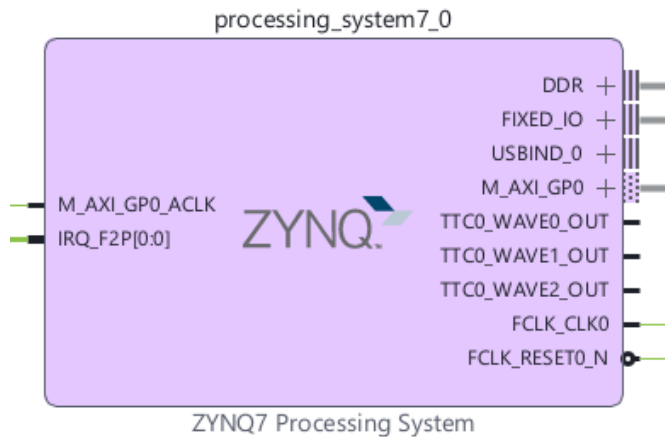
1. Design Flow

Giai đoạn	Đầu vào	Đầu ra và mục đích
Tạo project	Target, yêu cầu buttons/LED	Project chứa cấu hình thiết bị và nguồn thiết kế.
Tạo Block Design	PS, IP ngoại vi	Kiến trúc PS–PL và các cổng ngoài.
Tích hợp IP	AXI, clock, reset, IRQ	Kết nối và bản đồ địa chỉ cho ngoại vi.
Validate	Block Design	Kiểm tra cấu trúc/interface trước tổng hợp.
Wrapper, Synthesis	Block Design và HDL	Top-level và netlist logic.
Implementation	Netlist, constraints	Thiết kế đã place-and-route để đánh giá tài nguyên/timing.
Generate Bitstream	Implemented design	Sinh bitstream cho Programmable Logic (PL).
Export hardware	Hardware và bitstream	XSA bàn giao phần cứng sang Vitis.
Platform/BSP	XSA	Cấu hình processor, driver và metadata ngoại vi.
Application	Source C và BSP	Object, ELF; kiểm tra compile/link.
Mô phỏng hành vi	Mô hình và stimulus	Waveform kiểm tra logic mong đợi trong mô hình.

2. Các khối IP trong thiết kế

Instance	Loại IP	Vai trò
processing_system7_0	ZYNQ7 Processing System	AXI master, clock/reset, CPU và GIC.
axi_gpio_0	AXI GPIO	Buttons 5 bit, nguồn GPIO IRQ.
axi_gpio_1	AXI GPIO	LED output 8 bit.
axi_smc	AXI SmartConnect	Phân phối giao dịch tới ba slave.
rst_ps7_0_100M	Processor System Reset	Tạo reset cho bus/ngoại vi.
axi_timer_0	AXI Timer	Nguồn ngắt thời gian.
xlconcat_0	Concat	Ghép hai tín hiệu interrupt.

3. ZYNQ7 Processing System



Hình 1: ZYNQ7 Processing System trong ảnh thiết kế được cung cấp

Hình 1 trình bày `processing_system7_0`. Processing System (PS) thực thi chương trình C/C++; các giao dịch truy cập ngoại vi trong Programmable Logic (PL) được phát qua giao diện `M_AXI_GP0`. GIC nằm trong PS và tiếp nhận các yêu cầu ngắt từ PL. DDR và `FIXED_IO` là các giao diện vật lý của PS kết nối ra bo mạch.

4. Các cổng và cấu hình của `processing_system7_0`

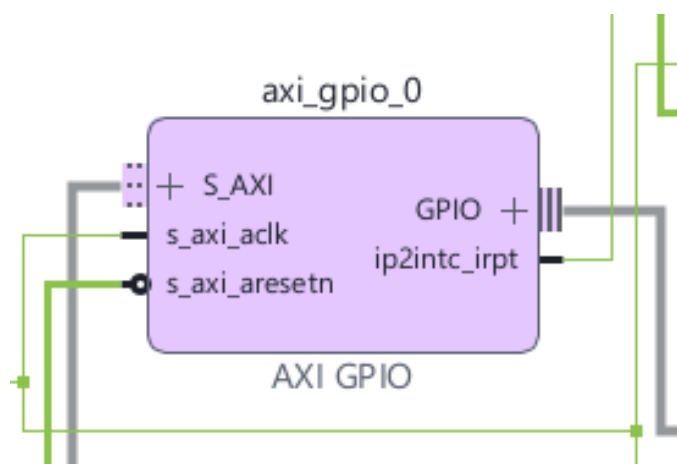
Interrupt Port	ID	Description
☒ Fabric Interrupts		Enable PL Interrupts to PS and vice versa
PL-PS Interrupt Ports		
☒ <code>IRQ_F2P[15:0]</code>	[91:84], [	Enables 16-bit shared interrupt port from the PL.
☐ <code>Core0_nFIQ</code>	28	Enables fast private interrupt signal for CPU0 from
☐ <code>Core0_nIRQ</code>	31	Enables private interrupt signal for CPU0 from the
☐ <code>Core1_nFIQ</code>	28	Enables fast private interrupt signal for CPU1 from
☐ <code>Core1_nIRQ</code>	31	Enables private interrupt signal for CPU1 from the

Hình 2: Cấu hình Fabric Interrupts của Zynq PS

Hình 2 cho thấy tùy chọn *Fabric Interrupts* và cổng `IRQ_F2P` trong cấu hình PS. Đây là bằng chứng cấu hình giao diện ngắt, chưa tự chứng minh bề rộng bus ở ảnh Block Design khác hoặc hoạt động ISR trên CPU.

Port	Interface / Signal type	Chức năng
DDR	Giao diện bộ nhớ DDR	Kết nối bộ nhớ DDR của PS ra board.
FIXED_IO	Giao diện I/O cố định của PS	Nhóm chân cố định của PS; không phải AXI GPIO PL.
M_AXI_GP0	Giao diện AXI Master	Khởi tạo giao dịch địa chỉ/dữ liệu tới các ngoại vi trong PL và nhận phản hồi AXI.
M_AXI_GP0_ACLK	Clock input	Clock cho giao diện M_AXI_GP0.
FCLK_CLK0	Clock output	Clock do PS cấp cho miền logic PL.
FCLK_RESET0_N	Active-low reset output	Tín hiệu reset của fabric, tác động mức thấp (active-low).
IRQ_F2P[1:0]	Interrupt input, 2 bit	Nhận hai interrupt sources từ PL qua xlconcat_0.
USBIND_0	Giao diện USB	USB-related interface của PS; không tham gia interrupt path của bài Lab.
TTC0_WAVE0_OUT	Output	Dạng sóng từ TTC0; không dùng để tạo IRQ Timer trong thiết kế này.
TTC0_WAVE1_OUT	Output	Ngõ ra waveform TTC0 thứ hai.
TTC0_WAVE2_OUT	Output	Ngõ ra waveform TTC0 thứ ba.

5. AXI GPIO đọc nút nhấn



Hình 3: Khối axi_gpio_0 nhận buttons và phát interrupt

Hình 3 là axi_gpio_0. IP này tiếp nhận trạng thái các nút nhấn tại cổng GPIO, ánh xạ dữ liệu vào các thanh ghi AXI4-Lite để PS có thể đọc và phát yêu cầu ngắt qua ip2intc_irpt. Phần mềm xử lý theo sự kiện ngắt thay vì liên tục thăm dò (polling) trạng thái nút nhấn trong vòng lặp chính.

Port	Interface / Signal type	Chức năng
S_AXI	AXI4-Lite Slave	Nhận giao dịch đọc/ghi thanh ghi GPIO từ PS qua SmartConnect.
s_axi_aclk	Clock input	Clock giao tiếp AXI.
s_axi_aresetn	Active-low reset input	Reset active-low.

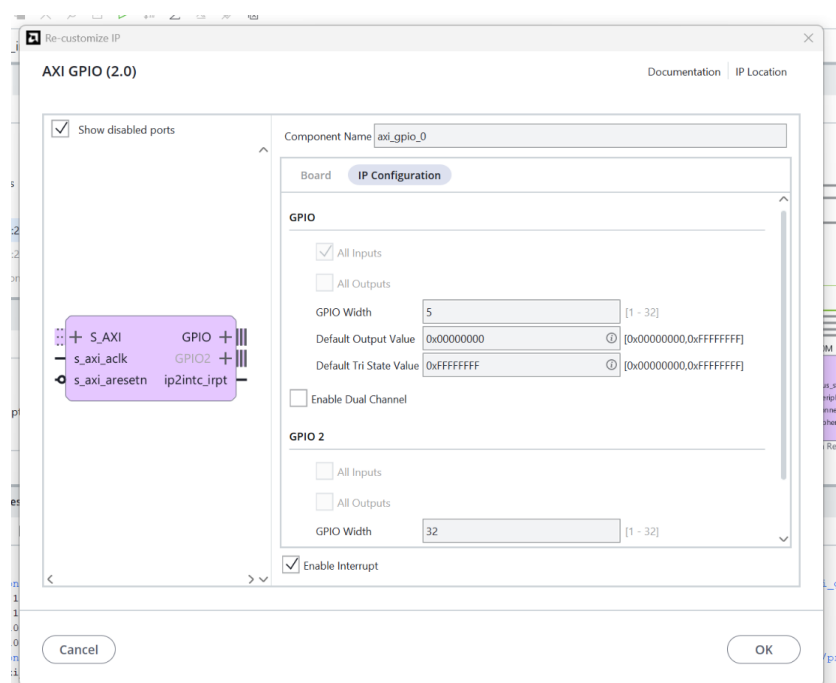
Port	Interface / Signal type	Chức năng
GPIO	GPIO input	Tiếp nhận trạng thái btnc_5bits theo vai trò của thiết kế.
ip2intc_irpt	Interrupt output	Yêu cầu ngắt mức cao từ GPIO.

Theo mô tả cổng AXI GPIO, ip2intc_irpt là tín hiệu ngắt nhạy mức. Vì vậy xung gpio_irq trong mô phỏng chức năng chỉ là biểu diễn của mô hình, không đủ xác nhận dạng sóng của IP thật. ISR cần đọc nguyên nhân và xử lý trạng thái ngắt của ngoại vi.

6. Quan hệ giữa dữ liệu GPIO và yêu cầu ngắt

Dữ liệu nút nhấn được đọc qua giao dịch AXI4-Lite, trong khi yêu cầu ngắt truyền trên một đường tín hiệu riêng. Khối Concat chỉ ghép các đường ngắt và không truyền giá trị nút nhấn tới CPU. Sau khi nhận ngắt, phần mềm dùng XGpio_DiscreteRead để đọc kênh GPIO rồi ghi dữ liệu kết quả ra AXI GPIO điều khiển LED.

7. Cấu hình AXI GPIO input và interrupt



Hình 4: Cấu hình GPIO cho buttons và Enable Interrupt

Hình 4 trình bày cửa sổ cấu hình GPIO. GPIO Width = 5 phù hợp năm bit của btnc_5bits; Enable Interrupt được chọn. Không dùng Dual Channel trong cấu hình được mô tả. Các ảnh thông số BSP ở phần kiểm chứng phần mềm cung cấp thêm GPIO_WIDTH = 0x5 và INTERRUPT_PRESENT = 0x1.

Thông tin	Bằng chứng	Ý nghĩa
Độ rộng GPIO	Width = 5; BSP 0x5	Một kênh gồm năm tín hiệu buttons.
Khả năng ngắt	Enable Interrupt; BSP 0x1	IP có phần thanh ghi/đường ngắt.
Cấu hình dữ liệu	Kênh GPIO ngõ vào; SetDataDirection	Phần mềm đặt kênh buttons là ngõ vào.
Cho phép ngắt khi chạy	Các lời gọi XGpio trong code	Được enable trong software bằng XGpio_InterruptEnable và XGpio_InterruptGlobalEnable.

Khả năng hỗ trợ interrupt của AXI GPIO và trạng thái enable interrupt khi software chạy là hai lớp khác nhau. XGpio_InterruptEnable/XGpio_InterruptGlobalEnable điều khiển phía GPIO; GIC và CPU exception handling là các lớp điều khiển tiếp theo trong interrupt path.

Không xác định thời gian chống dội hoặc số lần ngắt trên mỗi lần nhấn từ ảnh cấu hình này. Các thông số chống dội, nếu có, phải được chứng minh bằng thiết kế hoặc source thực tế, không bổ sung giả định vào báo cáo.

8. AXI GPIO điều khiển LED



Hình 5: Khối axi_gpio_1 điều khiển LED

Hình 5 là axi_gpio_1. IP nhận dữ liệu ghi qua AXI và đưa dữ liệu ra cổng GPIO nối leds_8bits. Khối này làm thiết bị hiển thị đầu ra; ảnh BSP xác nhận GPIO_WIDTH = 0x8 và INTERRUPT_PRESENT = 0x0.

Port	Interface / Signal type	Chức năng
S_AXI	AXI4-Lite Slave	Nhận dữ liệu ghi và các truy cập cấu hình từ PS.
s_axi_aclk	Clock input	Đồng bộ giao tiếp AXI.
s_axi_aresetn	Active-low reset input	Reset IP ở mức thấp.
GPIO	GPIO output	Đưa tám bit dữ liệu ra leds_8bits.

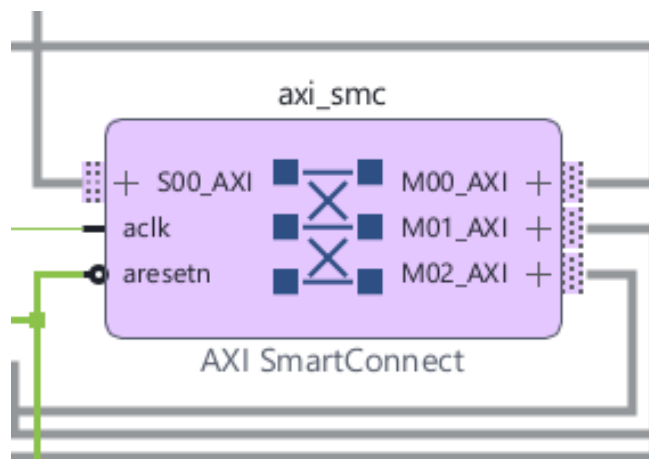
9. Quan hệ với chương trình điều khiển

main đặt hướng GPIO LED bằng XGpio_SetDataDirection(&LEDInst, 1, 0x00). Hai hàm xử lý ngắt đều có thể ghi giá trị led_data bằng XGpio_DiscreteWrite. Như vậy LED là điểm quan sát chung của sự kiện buttons và Timer, còn nguồn sự kiện được xác định qua đường interrupt tương ứng.

GPIO LED không cần phát ngắt để biểu diễn giá trị bộ đếm. Đây là sự khác biệt về nhiệm vụ so với GPIO buttons, dù hai khối đều sử dụng giao diện S_AXI, s_axi_aclk và s_axi_aresetn giống nhau.

Lớp quan sát	Kết quả có thể xác nhận
Mã nguồn C	Có lời gọi đặt hướng output và ghi dữ liệu LED.
Dạng sóng mô phỏng	Mô hình leds[7:0] thay đổi theo stimulus và bộ đếm mô phỏng.

10. AXI SmartConnect



Hình 6: AXI SmartConnect trong thiết kế

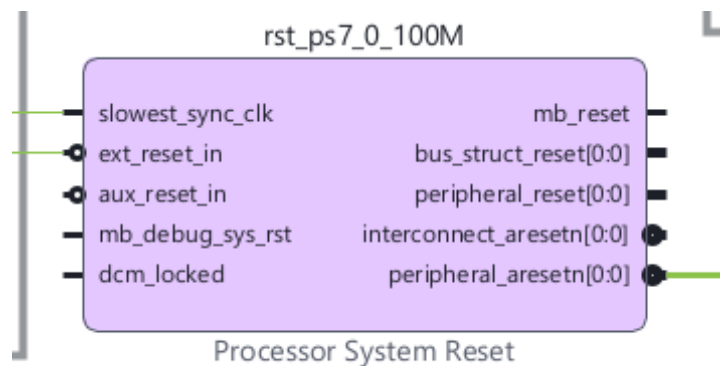
Hình 6 trình bày axi_smc với một giao diện slave S00_AXI và ba giao diện master M00_AXI, M01_AXI, M02_AXI. Cách gọi master/slave ở đây mô tả vai trò giao dịch của từng giao diện, không phải hướng của tất cả các dây thành phần trong giao diện.

Port	Interface / Signal type	Chức năng trong hệ thống
S00_AXI	AXI Slave interface	Tiếp nhận giao dịch từ M_AXI_GP0 của PS.
M00_AXI	AXI Master interface	Nhánh tới axi_gpio_0/S_AXI theo bảng kết nối bài làm.
M01_AXI	AXI Master interface	Nhánh tới axi_gpio_1/S_AXI theo bảng kết nối bài làm.
M02_AXI	AXI Master interface	Nhánh tới axi_timer_0/S_AXI theo bảng kết nối bài làm.
aclk	Clock input	Clock hoạt động của interconnect.
aresetn	Active-low reset input	Reset active-low.

PS khởi tạo các giao dịch đọc/ghi tới ngoại vi; SmartConnect định tuyến giao dịch đến giao diện AXI Slave tương ứng. GPIO và Timer dùng chung cổng AXI Master của PS nhưng được ánh xạ vào các vùng địa chỉ khác nhau. Phần kết nối giữa các khối mô tả đường liên kết, còn phần bản đồ địa chỉ và BSP mô tả các địa chỉ ngoại vi.

SmartConnect không xử lý ISR và không thay cho GIC. Nó thuộc đường truy cập thanh ghi; đường GPIO IRQ/Timer IRQ đi qua Concat sang PS là một đường riêng. Tách hai đường này giúp giải thích đúng vai trò của bus dữ liệu và tín hiệu báo sự kiện.

11. Processor System Reset



Hình 7: Các chân của rst_ps7_0_100M

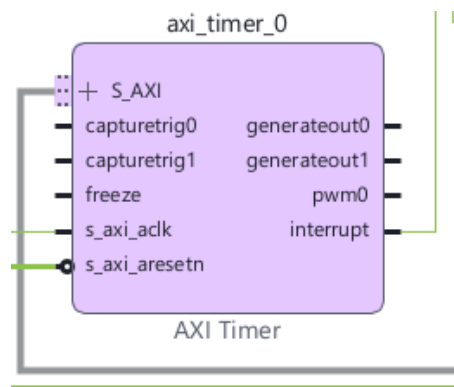
Hình 7 thể hiện các ngõ vào điều kiện reset và các nhóm ngõ ra reset. Tên instance có 100M nhưng tần số thực tế phải đối chiếu với clock/BSP, không xác nhận chỉ bằng tên khối.

Port	I/O	Chức năng
slowest_sync_clk	Input	Clock đồng bộ để tạo reset.
ext_reset_in	Input	Yêu cầu reset ngoài; cực tính cấu hình được.
aux_reset_in	Input	Nguồn reset phụ; cực tính cấu hình được.
mb_debug_sys_rst	Input	Yêu cầu reset debug MicroBlaze, active-high.
dcm_locked	Input	Thông báo clock đã khóa/ổn định.
mb_reset	Output	Reset core MicroBlaze, active-high.
bus_struct_reset[0:0]	Output	Reset cấu trúc bus, active-high.
peripheral_reset[0:0]	Output	Reset ngoại vi, active-high.
interconnect_aresetn[0:0]	Output	Reset interconnect, active-low.
peripheral_aresetn[0:0]	Output	Reset ngoại vi, active-low.

Ảnh cho thấy peripheral_aresetn được phân phối cho miền AXI; interconnect_aresetn không có đường nối nhìn thấy. Các chân không thấy đường nối không được gán tùy ý giá trị 0 hoặc 1 trong bảng kết nối.

Trong thiết kế này, slowest_sync_clk nhận FCLK_CLK0 và peripheral_aresetn được dùng làm active-low reset cho các AXI peripherals. Các port còn lại của Processor System Reset không tham gia trực tiếp vào datapath/interrupt path được phân tích trong bài Lab.

12. AXI Timer



Hình 8: AXI Timer tạo nguồn ngắt định kỳ

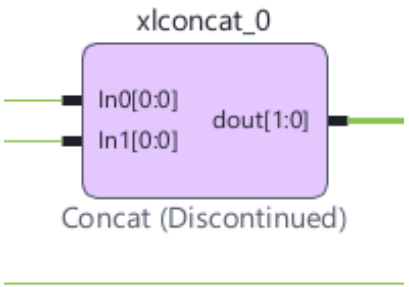
Hình 8 là `axi_timer_0`. IP có giao diện thanh ghi AXI4-Lite, các cổng capture/generate/PWM và đầu ra interrupt. Bài Lab sử dụng nguồn ngắt Timer; không có bằng chứng sử dụng các cổng tạo PWM hoặc capture cho chức năng buttons–LED.

Port	Interface / Signal type	Chức năng
S_AXI	AXI4-Lite Slave	Truy cập thanh ghi điều khiển và bộ đếm.
s_axi_aclk	Clock input	Clock AXI của IP.
s_axi_aresetn	Active-low reset input	Reset active-low.
capturetrig0	Input	Trigger capture của Timer 0.
capturetrig1	Input	Trigger capture của Timer 1.
freeze	Input	Điều khiển đóng băng bộ đếm theo chức năng IP.
generateout0	Output	Tín hiệu generate của Timer 0.
generateout1	Output	Tín hiệu generate của Timer 1.
pwm0	Output	Ngõ ra PWM.
interrupt	Output	Yêu cầu ngắt do điều kiện Timer.

Trong mã nguồn có `TMR_LOAD = 0xF8000000`, `XTC_INT_MODE_OPTION` và `XTC_AUTO_RELOAD_OPTION`. Đây là các tham số cấu hình phần mềm, không phải thông số đọc từ cửa sổ Customize IP của AXI Timer.

Ảnh BSP cho thấy clock 100 MHz. Chu kỳ `timer_irq` rút ngắn trên waveform là thông số của mô hình mô phỏng; không được xem là chu kỳ đo trên AXI Timer thật với giá trị nạp ở code C.

13. Concat (xlconcat_0)



Hình 9: Concat ghép hai tín hiệu ngắt

Hình 9 cho thấy In0[0:0], In1[0:0] và dout[1:0]. Concat ghép các bit thành bus; không lựa chọn ưu tiên, không acknowledge ngắt và không chạy ISR. GIC trong PS mới thực hiện quản lý ngắt cho CPU.

Port	I/O	Ý nghĩa
In0[0:0]	Input	Nhận một bit GPIO interrupt theo kết nối bài làm.
In1[0:0]	Input	Nhận một bit Timer interrupt theo kết nối bài làm.
dout[1:0]	Output	Bus hai bit giữ riêng hai nguồn.

Component Name

Number of Ports [1 - 128]

☒ In0 Width

[1 - 4096]

☒ In1 Width

[1 - 4096]

Dout Width (Auto)

NOTE: The In0 port is connected to the LSB bits of the output, and the In[Number of Ports - 1] input port is connected to the MSB bits of the output.

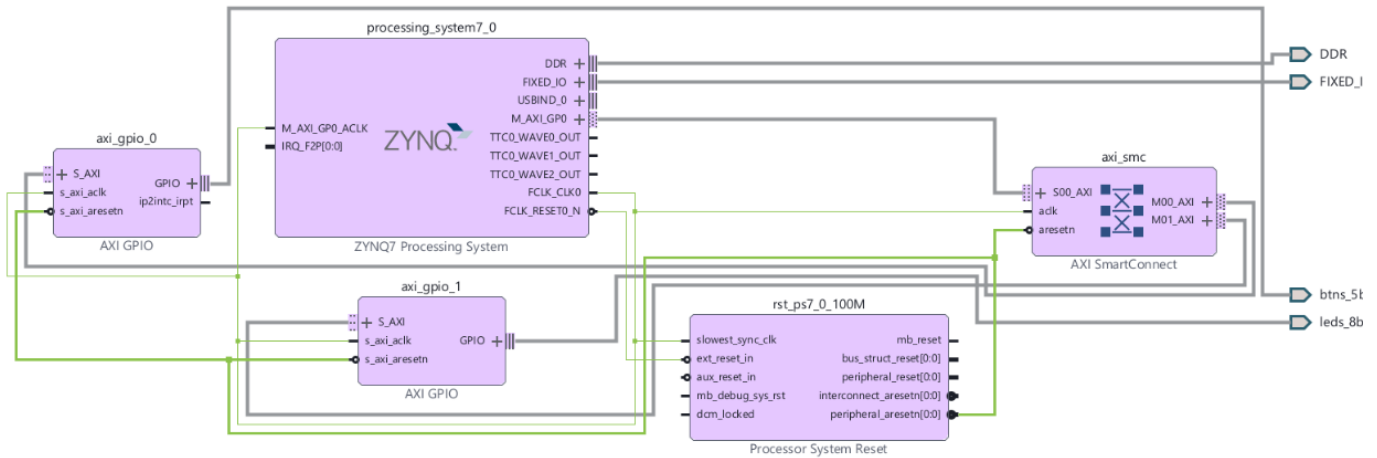
Hình 10: Cấu hình số cổng và độ rộng của Concat

Hình 10 ghi Number of Ports = 2, In0 Width = 1, In1 Width = 1 và Dout Width = 2. Ghép hai nguồn độc lập cho phép phía nhận phân biệt vị trí bit; một phép OR hai nguồn sẽ không giữ được cùng thông tin nguồn trên đường tín hiệu.

Block Design cuối xác nhận xlconcat_0/dout[1:0] nối với processing_system7_0/IRQ_F2P[1:0], phù hợp hai interrupt sources độc lập từ AXI GPIO và AXI Timer.

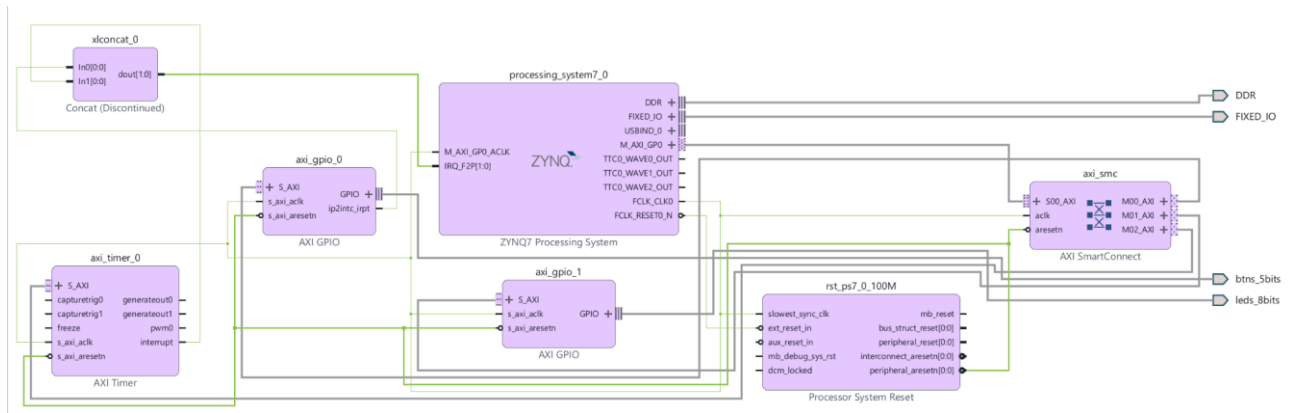
14. Kết nối giữa các khối

15. Block Design cơ sở và Block Design mở rộng



Hình 11: Block Design cơ sở trong giai đoạn cấu hình GPIO

Hình 11 thể hiện cấu hình cơ sở gồm PS, hai GPIO, interconnect và reset. Các cổng ip2intc_irpt và IRQ_F2P có xuất hiện nhưng chưa thấy đường nối giữa chúng trong ảnh. Vì vậy ảnh này chỉ mô tả giai đoạn cấu hình cơ sở; không dùng để xác nhận đường ngắt đã nối hoàn tất hoặc hoạt động Timer.

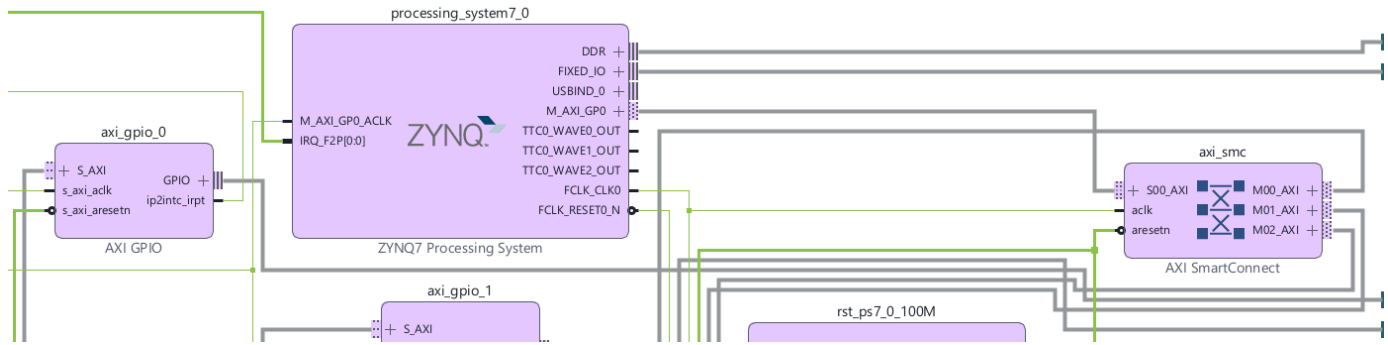


Hình 12: Block Design cuối của hệ thống

Hình 12 trình bày Block Design cuối của hệ thống, gồm processing_system7_0, hai AXI GPIO, AXI Timer, AXI SmartConnect, Processor System Reset và xlconcat_0. SmartConnect cung cấp ba AXI Master interfaces cho hai GPIO và Timer. Đường interrupt được ghép qua xlconcat_0 và đưa vào IRQ_F2P[1:0] của Processing System.

Trong Block Design cuối, xlconcat_0/dout[1:0] nối trực tiếp với processing_system7_0/IRQ_F2P[1:0], do đó bề rộng interrupt bus giữa PL và PS là 2 bit và phù hợp với hai nguồn interrupt độc lập.

16. Kết nối AXI và GPIO



Hình 13: Vùng kết nối AXI từ PS qua SmartConnect

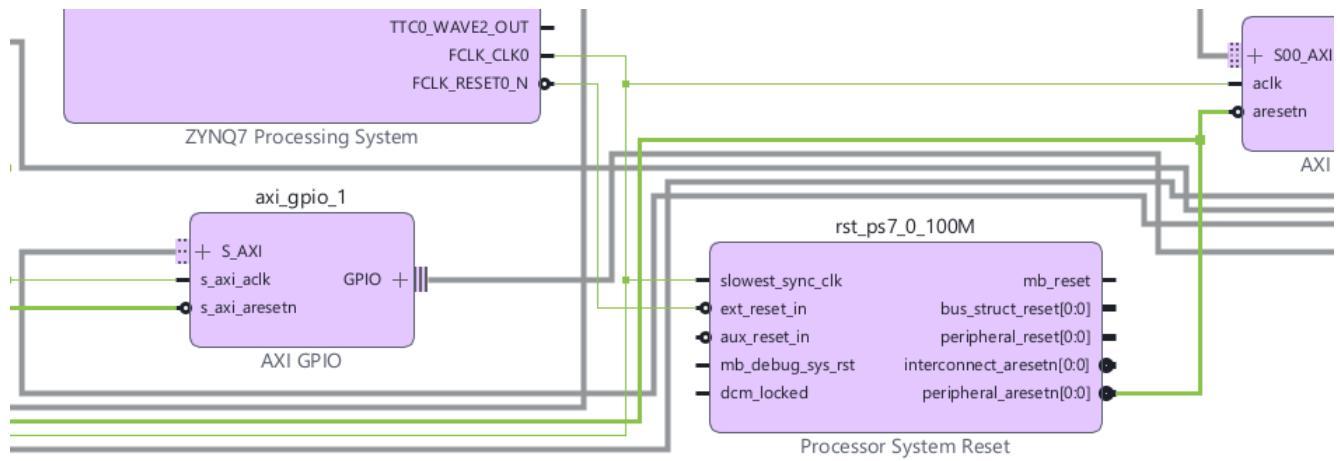
Hình 13 cho thấy đường giao dịch memory-mapped. Các giao diện AXI có kênh yêu cầu và phản hồi, vì vậy nguồn–đích trong bảng được hiểu theo phía khởi tạo giao dịch. Hai dòng GPIO ngoài được ghi theo hướng dữ liệu vào/ra của ứng dụng.

Nguồn	Đích	Loại	Ý nghĩa
PS / M_AXI_GP0	axi_smc / S00_AXI	AXI	PS truy cập AXI peripheral trong PL.
axi_smc / M00_AXI	axi_gpio_0 / S_AXI	AXI4-Lite phía slave	Đọc buttons và cấu hình GPIO.
axi_smc / M01_AXI	axi_gpio_1 / S_AXI	AXI4-Lite phía slave	Ghi dữ liệu LED.
axi_smc / M02_AXI	axi_timer_0 / S_AXI	AXI4-Lite phía slave	Cấu hình Timer.
Cổng ngoài btns_5bits	axi_gpio_0 / GPIO	GPIO input, 5 bit	Đưa trạng thái buttons vào IP.
axi_gpio_1 / GPIO	Cổng ngoài leds_8bits	GPIO output, 8 bit	Xuất dữ liệu LED.
PS / DDR	Cổng ngoài DDR	Giao diện DDR	Kết nối bộ nhớ của PS; interface hỗn hợp.
PS / FIXED_IO	Cổng ngoài FIXED_IO	PS fixed I/O	Nhóm chân cố định; interface hỗn hợp.

Mỗi giao dịch GPIO/Timer phải thuộc vùng địa chỉ đã được bàn giao trong XSA. Bảng này không suy ra địa chỉ từ vị trí hình vẽ; các địa chỉ được lấy từ xparameters.h và trình bày trong phần bản đồ địa chỉ cùng các thông số BSP.

Các AXI interfaces được đối chiếu trên Block Design cuối: M_AXI_GP0 của PS nối tới S00_AXI của SmartConnect; các master interfaces M00_AXI–M02_AXI được dùng để truy cập hai AXI GPIO và AXI Timer. Mỗi peripheral có một vùng địa chỉ riêng trong Address Editor.

17. Kết nối Clock và Reset



Hình 14: Vùng phân phối clock và reset

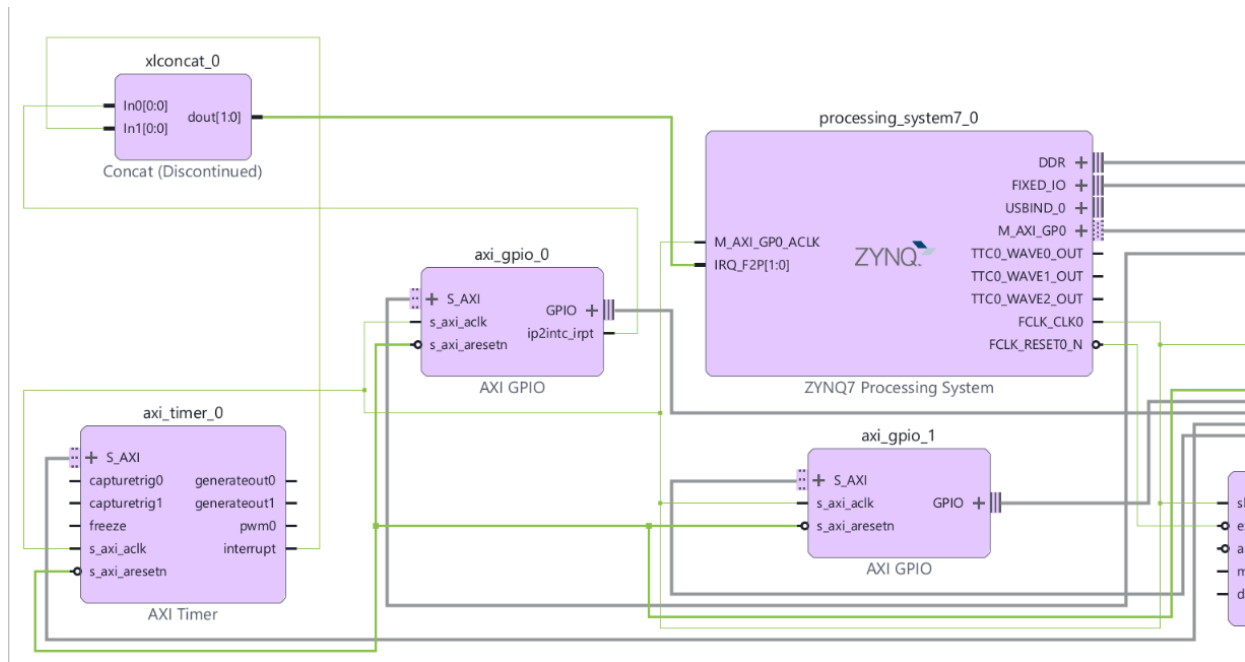
Hình 14 trình bày miền clock/reset của hệ AXI. Clock cho giao diện master của PS phải được cung cấp bên cạnh clock cho SmartConnect và các slave. Reset của từng khối cần đúng cực tính và phù hợp với clock sử dụng.

Nguồn	Đích	Loại	Ý nghĩa
PS / FCLK_CLK0	PS / M_AXI_GP0_ACLK	Clock	Clock cho master AXI.
PS / FCLK_CLK0	axi_smc / aclk	Clock	Clock interconnect.
PS / FCLK_CLK0	axi_gpio_0, axi_gpio_1, axi_timer_0 / s_axi_aclk	Clock	Clock các slave.
PS / FCLK_CLK0	rst_ps7_0_100M / slowest_sync_clk	Clock	Đồng bộ khối reset.
PS / FCLK_RESET0_N	rst_ps7_0_100M / ext_reset_in	Reset	Nguồn yêu cầu reset từ PS.
rst_ps7_0_100M / peripheral_aresetn	axi_smc / aresetn	Active-low	Reset SmartConnect theo đường nối trong ảnh.
rst_ps7_0_100M / peripheral_aresetn	GPIO/Timer / s_axi_aresetn	Active-low	Reset các peripheral.

Ảnh Timer BSP ghi CLOCK_FREQUENCY = 0x5F5E100, tương đương 100 MHz. Đây là bảng chứng metadata clock cho Timer, không thay thế Clock Summary hoặc constraints của toàn bộ thiết kế.

Hình 14 cho thấy FCLK_CLK0 được phân phối tới AXI clock inputs và slowest_sync_clk; peripheral_aresetn của Processor System Reset được phân phối tới SmartConnect và các AXI peripherals dưới dạng active-low reset.

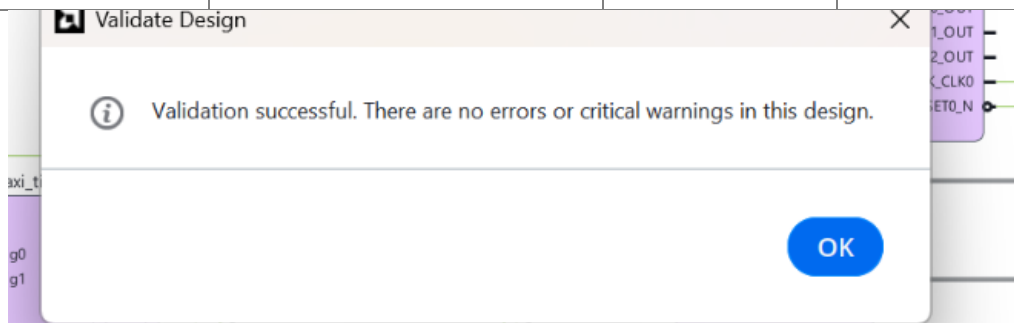
18. Interrupt Path và Validate Design



Hình 15: Interrupt path từ AXI GPIO và AXI Timer qua Concat tới PS

Hình 15 thể hiện interrupt path trong PL. axi_gpio_0/ip2intc_irpt nối vào xlconcat_0/In0, axi_timer_0/interrupt nối vào xlconcat_0/In1; xlconcat_0/dout[1:0] nối tới processing_system7_0/IRQ_F2P[1:0]. Dữ liệu buttons không truyền trên interrupt bus mà được CPU đọc qua AXI4-Lite sau khi nhận interrupt.

Nguồn	Đích	Loại	Ý nghĩa
axi_gpio_0 / ip2intc_irpt	xlconcat_0 / In0	Interrupt, 1 bit	Nguồn buttons / GPIO.
axi_timer_0 / interrupt	xlconcat_0 / In1	Interrupt, 1 bit	Nguồn Timer.
xlconcat_0 / dout[1:0]	PS / IRQ_F2P[1:0]	Interrupt bus, 2 bit	Hai bit giữ riêng nguồn GPIO và Timer.



Hình 16: Thông báo Validate Design thành công

Hình 16 cho thấy Validate Design hoàn tất thành công, không có error hoặc critical warning trong lần kiểm tra IP Integrator đó. Kết quả này không đồng nghĩa toàn bộ quá trình Implementation không có cảnh báo; cửa sổ Design Runs vẫn ghi 13 Methodology Warnings.

Validate Design xác nhận tính nhất quán cấu trúc của Block Design trong IP Integrator. Synthesis, Implementation và Timing được đánh giá riêng bằng các Design Reports ở các phần tiếp theo.

19. Address Map và tính nhất quán Hardware/Software

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/processing_system7_0/Data)					
/processing_system7_0					
/processing_system7_0/Data (32 address bits : 0x40000000 [1G])					
/axi_gpio_0/S_AXI	S_AXI	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_gpio_1/S_AXI	S_AXI	Reg	0x4121_0000	64K	0x4121_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x4280_0000	64K	0x4280_FFFF

Hình 17: Address Editor của project cuối

Hình 17 xác nhận Address Map của các AXI peripherals trong project cuối. axi_gpio_0/S_AXI được ánh xạ tại 0x4120_0000–0x4120_FFFF, axi_gpio_1/S_AXI tại 0x4121_0000–0x4121_FFFF và axi_timer_0/S_AXI tại 0x4280_0000–0x4280_FFFF; mỗi vùng có Range = 64K.

Peripheral	Base Address	High Address	Evidence
AXI GPIO 0	0x41200000	0x4120FFFF	Hình 17 – Address Editor
AXI GPIO 1	0x41210000	0x4121FFFF	Hình 17 – Address Editor
AXI Timer 0	0x42800000	0x4280FFFF	Hình 17 – Address Editor
INTC trong BSP	0xF8F01000	0xF8F01FFF	BSP / xparameters.h

Các địa chỉ GPIO và Timer phân biệt các peripheral trong không gian memory-mapped. Hai GPIO cùng loại IP nhưng khác vùng địa chỉ và chức năng. Sai platform/BSP có thể làm ứng dụng sử dụng metadata không tương ứng với hardware; vì vậy ảnh cập nhật XSA là một phần cần thiết của chuỗi bằng chứng.

20. Đối chiếu định danh ngắt

Ảnh BSP ghi `XPAR_FABRIC_AXI_GPIO_0_INTR = 29` và `XPAR_FABRIC_AXI_TIMER_0_INTR = 30`. Code trong Word lại dùng các macro mang tên `XPAR_FABRIC_AXI_GPIO_0_IP2INTC_IRPT_INTR` và `XPAR_FABRIC_AXI_TIMER_0_INTERRUPT_INTR`. Các tên không trùng nhau trên những đoạn ảnh hiện có.

Interrupt IDs phải sử dụng đúng macro do BSP/xparameters.h của platform hiện tại sinh ra. Trong report này, các giá trị 29 và 30 được dùng để đối chiếu hai fabric interrupts tương ứng với AXI GPIO và AXI Timer.

21. Đối chiếu Address Map cuối

Nội dung	Kết quả
IP instance và base address	Đã xác nhận bằng Address Editor của project cuối.
XSA tương ứng với ảnh Block Design	Block Design cuối xác nhận IRQ_F2P[1:0] và bus Concat 2 bit; XSA được cập nhật từ hardware design.
Tên macro và API driver	Driver/API sử dụng theo BSP của platform; các interrupt IDs được lấy từ xparameters.h.
Address Editor của project cuối	Đã xác nhận tại Hình 17.

Các địa chỉ trong bảng được lấy từ thông tin BSP/xparameters.h của bài làm và dùng để đối chiếu không gian địa chỉ memory-mapped của các ngoại vi.

22. Synthesis, Implementation và Design Reports

23. Synthesis và Implementation Status

me	Constraints	Status
✓ synth_1 (active)	constrs_1	synth_design Complete!

Hình 18: Trạng thái synth_1 trong Design Runs

Hình 18 cho thấy synth_1 có trạng thái synth_design Complete!. Nguồn là bảng Design Runs trong Vivado. Synthesis chuyển mô tả HDL/IP thành netlist phục vụ các giai đoạn triển khai; hoàn tất synthesis chưa xác nhận timing sau định tuyến.

me	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	!
✓ synth_1 (active)	constrs_1	synth_design Complete!												0	0	0	0	0	
✓ impl_1	constrs_1	write_bitstream Complete!	3.522	0.000	0.106	0.000		0.000	1.684	0	13 Warn			855	912	0	0	0	

Hình 19: Trạng thái impl_1 và các cột kết quả trong Design Runs

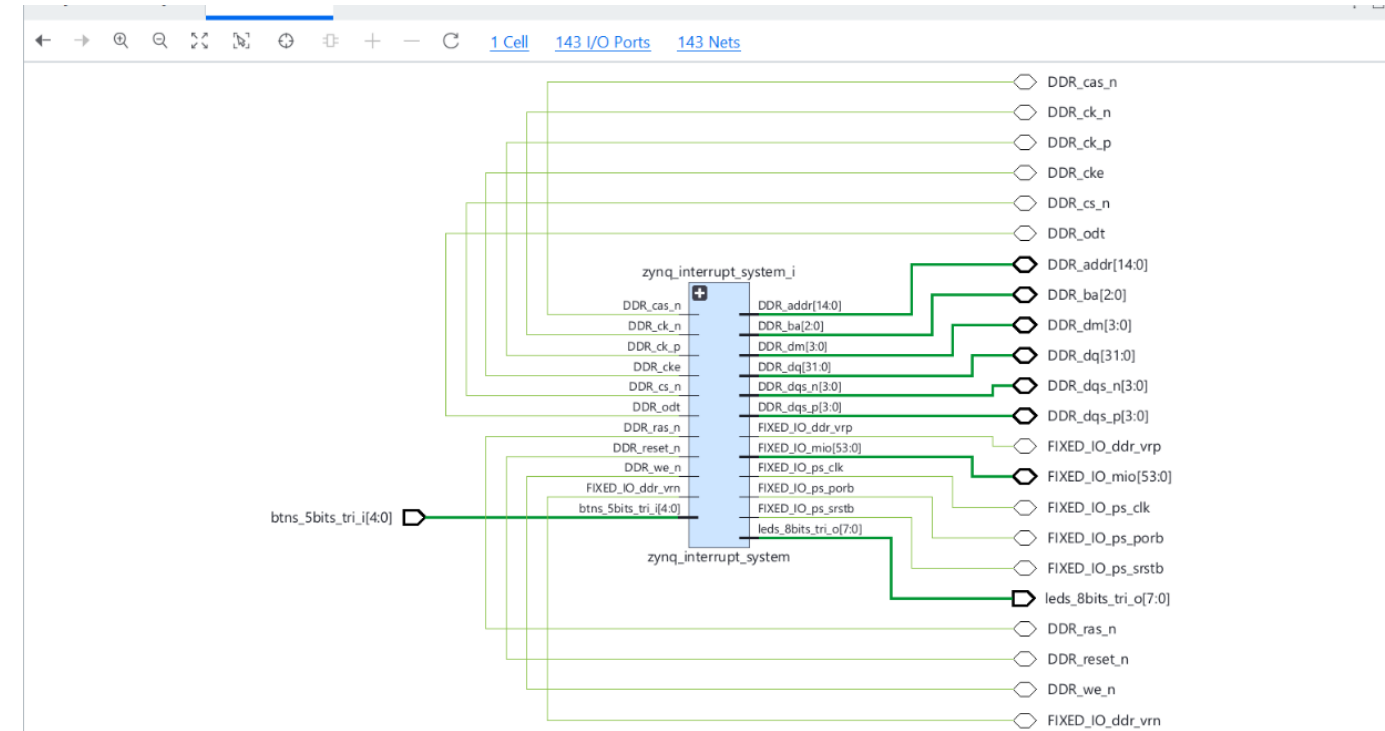
Hình 19 ghi impl_1 ở trạng thái write_bitstream Complete!, Failed Routes = 0 và Total Power = 1.684 W. Các cột WNS/WHS trong Design Runs là snapshot của run; kết quả Timing được chốt theo Design Timing Summary ở Hình 22.

Quan sát	Ý nghĩa và giới hạn
synth_design Complete!	Có bằng chứng run synthesis hoàn tất.
write_bitstream Complete!	Có bằng chứng run triển khai đã đến bước tạo bitstream.
Failed Routes = 0	Không có failed routes trong dòng run được chụp.
Methodology = 13 Warn	TIMING-18: 13 warnings, gồm 5 input ports thiếu input delay và 8 output ports thiếu output delay.
Các cột tài nguyên của synth_1 có giá trị 0	Không dùng các ô này để kết luận toàn thiết kế không dùng tài nguyên; đối chiếu Utilization và impl_1.

13 Methodology Warnings được xác định là TIMING-18: Missing input or output delay, gồm 5 input ports và 8 output ports chưa khai báo I/O delay. Cảnh báo này được giữ trong phần Timing để phân biệt với lỗi Synthesis/Implementation.

Các trạng thái run được đối chiếu với Design Runs; Timing và Resource Utilization được xác nhận thêm bằng report_timing_summary và report_utilization trên routed design.

24. Schematic



Hình 20: Schematic top-level của zynq_interrupt_system_wrapper

Hình 20 là sơ đồ schematic ở cấp wrapper. Các cổng buttons, LEDs, DDR và FIXED_IO xuất hiện quanh khối thiết kế. Sơ đồ này mô tả kết nối logic/netlist ở mức đang mở, trong khi Block Design trình bày các IP chức năng trước đó.

Thành phần nhìn thấy	Liên hệ với Block Design
zynq_interrupt_system_wrapper	Top-level chứa hệ thống IP Integrator.
Nhóm DDR và FIXED_IO	Tương ứng giao diện PS ra board.
Buttons	Tương ứng cổng ngoài dẫn tới GPIO input.
LEDs	Tương ứng cổng ngoài của GPIO output.
Các nhánh/buffer ở biên	Thể hiện cấu trúc kết nối I/O ở schematic; không phải một IP ngoại vi mới được thêm vào phần mô tả các khối IP.

Hình 20 là Schematic top-level của `zynq_interrupt_system_wrapper`. Schematic được dùng để quan sát cấu trúc netlist và các top-level ports; phần phân tích chức năng từng IP vẫn dựa trên Block Design. Ảnh top-level chưa mở hierarchy để nhìn thấy logic bên trong từng GPIO/Timer. Do đó không dùng schematic này để kết luận số LUT/FF của mỗi IP, độ trễ của ISR hoặc tính đúng đắn của các thanh ghi phần mềm.

25. Resource Utilization

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (200)	Bonded IOPADs (130)	BUFGCTRL (32)
zynq_interrupt_system_wrapper	855	912	1	328	854	1	13	130	1
zynq_interrupt_system_i (zynq_interrupt_system)	855	912	1	328	854	1	0	0	1

Hình 21: Bảng Utilization theo hierarchy wrapper và system

Hình 21 và report_utilization của routed design cho thấy thiết kế sử dụng 855 Slice LUTs (1.61%), 912 Slice Registers (0.86%), 328 Slices (2.47%), 13 Bonded IOB (6.50%) và 1 BUFGCTRL (3.13%). Block RAM Tile và DSP đều không được sử dụng.

Tài nguyên	Used	Available	Tỷ lệ tính từ ảnh
Slice LUTs	855	53,200	1.61%
Slice Registers	912	106,400	0.86%
F7 Muxes	1	26,600	<0.01%
Slice	328	13,300	2.47%
LUT as Logic	854	53,200	1.61%
LUT as Memory	1	17,400	<0.01%
Bonded IOB	13	200	6.50%
Bonded IOPADs	130	130	100.00%
BUFGCTRL	1	32	3.13%
Block RAM Tile	0	140	0.00%
DSPs	0	220	0.00%

Các phần trăm có tổng rõ ràng được tính bằng $\text{Used/Available} \times 100$ và làm tròn. LUT as Logic và LUT as Memory là các phân loại của LUT, không được cộng thêm vào Slice LUTs như những tài nguyên độc lập.

Dòng system_i có cùng số LUT/FF với wrapper nhưng Bonded IOB và IOPADs bằng 0 ở hierarchy này. Do đó không cộng hai dòng hierarchy để tính tổng thiết kế. Mức LUT 1.61% và Register 0.86% cho thấy phần logic PL được báo cáo nhỏ so với khả năng thiết bị; chưa chứng minh còn dư timing hoặc không có vấn đề constraints.

Resource Utilization ở mức top-level đủ để đánh giá mức sử dụng tài nguyên của thiết kế. Báo cáo không cộng các dòng hierarchy với nhau và không phân bổ LUT/Register cho từng IP khi không có hierarchy breakdown tương ứng.

26. Timing

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	3.522 ns	Worst Hold Slack (WHS)	0.106 ns	Worst Pulse Width Slack (WPWS)	4.020 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	2247	Total Number of Endpoints	2247	Total Number of Endpoints	914
All user specified timing constraints are met.					

Hình 22: Design Timing Summary của thiết kế sau Implementation

Hình 22 là Design Timing Summary của routed design sau Implementation. Kết quả: WNS = 3.522 ns, TNS = 0.000 ns; WHS = 0.106 ns, THS = 0.000 ns; WPWS = 4.020 ns, TPWS = 0.000 ns. Setup, Hold và Pulse Width đều có 0 failing endpoint. Vivado kết luận “All user specified timing constraints are met.”

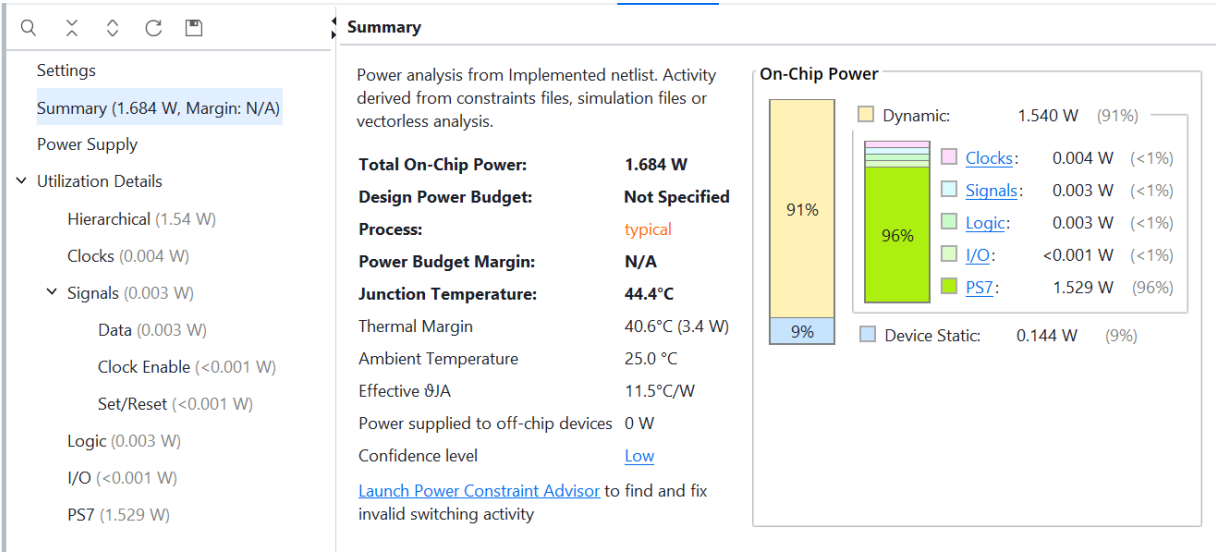
Chỉ số	Giá trị hiện có	Diễn giải
WNS	3.522 ns	Worst Negative Slack của Setup; giá trị dương.
TNS	0.000 ns	Total Negative Slack của Setup bằng 0.
WHS	0.106 ns	Worst Hold Slack; giá trị dương.
THS	0.000 ns	Total Hold Slack bằng 0.
WPWS	4.020 ns	Worst Pulse Width Slack; giá trị dương.
TPWS	0.000 ns	Total Pulse Width Negative Slack bằng 0.
Failing endpoints	0	Không có failing endpoint ở Setup, Hold và Pulse Width.
Total endpoints	2247 (Setup/Hold); 914 (Pulse Width)	Số endpoints được Timing Summary kiểm tra.
Timing constraints	MET	Vivado: “All user specified timing constraints are met.”

WNS, WHS và WPWS đều dương; TNS, THS và TPWS đều bằng 0.000 ns. Clock Summary cho clk_fpga_0 có Period = 10.000 ns, tương ứng Frequency = 100 MHz.

Timing Summary xác nhận thiết kế đáp ứng clock constraint 100 MHz đang khai báo. WNS không được dùng riêng lẻ để suy ra maximum operating frequency nếu chưa phân tích đầy đủ critical path và các constraints liên quan.

Report Methodology ghi TIMING-18 (Missing input or output delay) với 13 violations: 5 input ports chưa có input delay và 8 output ports chưa có output delay. Đồng thời check_timing không ghi nhận unconstrained internal endpoints hay combinational loops. Vì vậy, kết luận timing met áp dụng cho các timing constraints đã khai báo; I/O delay constraints vẫn là một cảnh báo methodology cần được nêu rõ.

27. Power Report



Hình 23: Power Summary và phân bố công suất trong ảnh bài làm

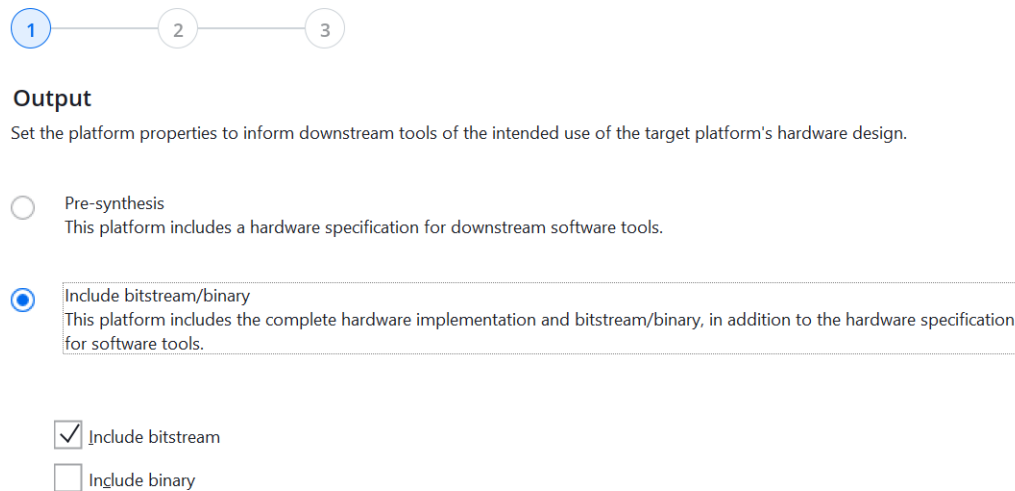
Hình 23 là Power Report từ implemented netlist. Total On-Chip Power = 1.684 W, gồm Dynamic = 1.540 W (91%) và Device Static = 0.144 W (9%). Giao diện Vivado hiển thị PS7 = 1.529 W (96%), Junction Temperature = 44.4 °C, Ambient Temperature = 25.0 °C và Thermal Margin = 40.6 °C (3.4 W). Confidence level của power estimate là Low.

Thông số	Giá trị
Total On-Chip Power	1.684 W
Dynamic	1.540 W; hiển thị 91%
Device Static	0.144 W; hiển thị 9%
PS7	1.529 W; Vivado hiển thị 96%
Clocks / Signals / Logic	0.004 W / 0.003 W / 0.003 W
I/O	<0.001 W
Junction / Ambient Temperature	44.4 °C / 25.0 °C
Thermal Margin	40.6 °C (3.4 W)
Process	typical
Confidence level	Low

PS7 là thành phần chiếm ưu thế trong power breakdown hiển thị. Confidence level = Low cho biết switching activity chủ yếu dựa trên constraints/vectorless analysis; vì vậy số liệu được sử dụng như power estimate của Vivado, không phải giá trị đo thực nghiệm.

28. Software Flow trên Processing System

29. Export Hardware và bàn giao XSA



Output

Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design.

☐ Pre-synthesis
This platform includes a hardware specification for downstream software tools.

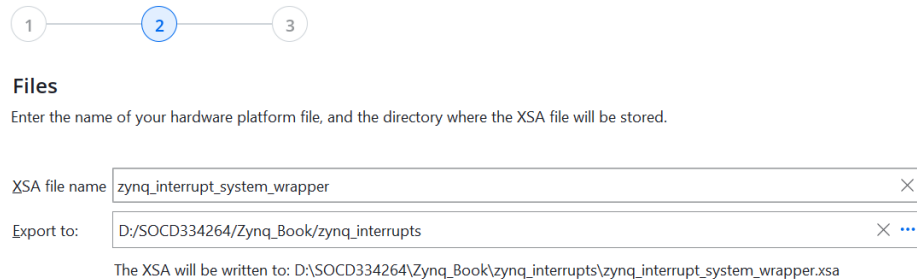
☒ Include bitstream/binary
This platform includes the complete hardware implementation and bitstream/binary, in addition to the hardware specification for software tools.

☒ Include bitstream

☐ Include binary

Hình 24: Tùy chọn export hardware có kèm bitstream

Hình 24 cho thấy tùy chọn Include bitstream được chọn khi Export Hardware. XSA chứa hardware specification cùng thông tin bitstream để Vitis tạo/cập nhật platform.



Files

Enter the name of your hardware platform file, and the directory where the XSA file will be stored.

XSA file name:

Export to:

The XSA will be written to: D:/SOC334264/Zynq_Book/zynq_interrupts/zynq_interrupt_system_wrapper.xsa

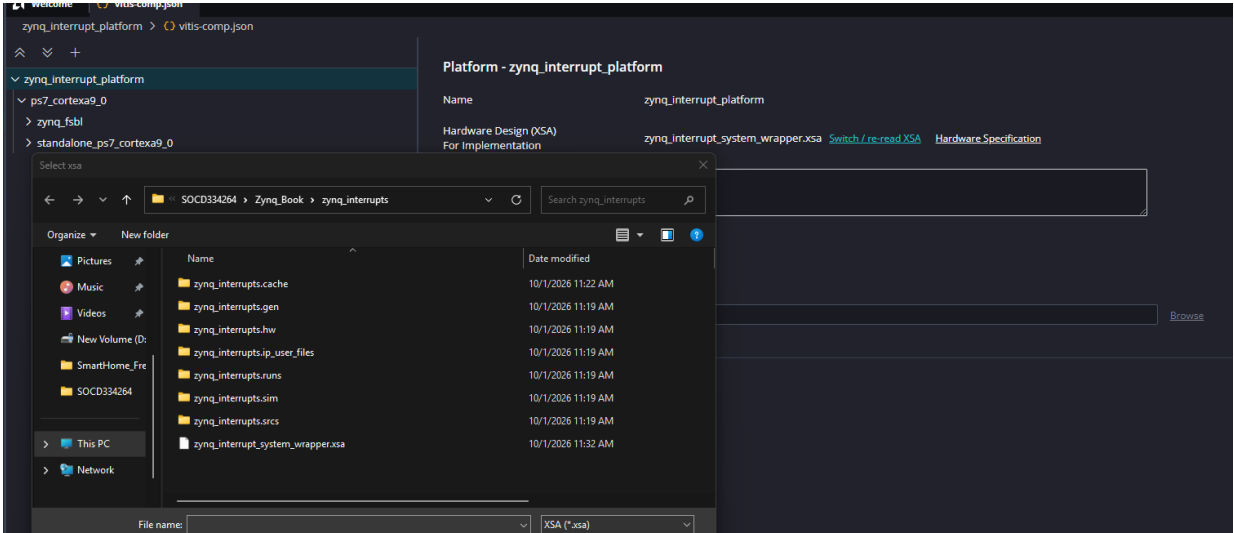
Hình 25: Tên và vị trí XSA được export

Hình 25 cho thấy file `zynq_interrupt_system_wrapper.xsa` được export từ hardware design hiện tại. XSA này được dùng để cập nhật platform trước khi build application.

Platform/BSP sử dụng metadata trong XSA để tạo driver parameters, base addresses và interrupt definitions cho software. Sau khi thêm AXI Timer và Concat, platform được cập nhật lại từ XSA mới ở Hình 26–27.

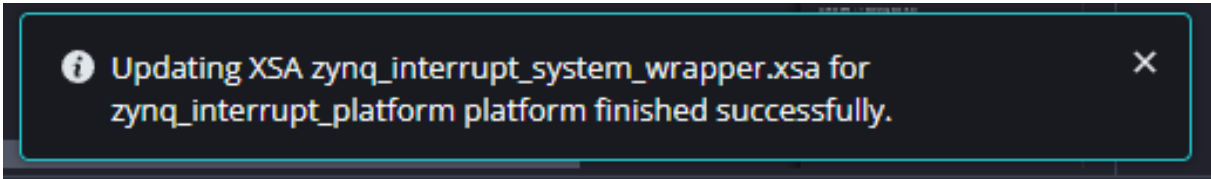
Tên XSA và giao diện Vitis trong báo cáo được lấy từ bài làm hiện tại. Báo cáo sử dụng thuật ngữ của Vitis và không quy đổi sang giao diện SDK cũ.

30. Cập nhật XSA và Build Platform



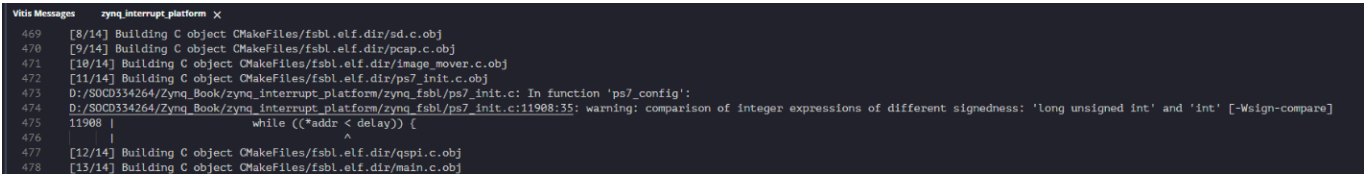
Hình 26: Cửa sổ cập nhật hardware platform trong Vitis

Hình 26 trình bày platform `zynq_interrupt_platform` và cửa sổ chọn hardware. Ảnh này cho biết đối tượng đang được cập nhật; kết quả cập nhật được xác nhận bằng thông báo riêng ở Hình 27.



Hình 27: Thông báo cập nhật XSA thành công

Hình 27 ghi việc cập nhật XSA của `zynq_interrupt_system_wrapper` cho `zynq_interrupt_platform` đã hoàn tất thành công. Đây là bằng chứng của thao tác cập nhật platform, không phải kết quả chạy application.



Hình 28: Log build platform trong Vitis

Hình 28 thể hiện quá trình build FSBL/platform. Cảnh báo signedness trong `ps7_init.c` thuộc phần code khởi tạo được sinh tự động và được ghi nhận như warning; không có bằng chứng trong ảnh cho thấy đây là lỗi làm dừng quá trình build.

Artifact	Vai trò
XSA	Đặc tả hardware đầu vào cho platform.
Platform/BSP	Driver và cấu hình ngoại vi cho processor đích.
Application ELF	Executable artifact được linker tạo từ application source và BSP.

Cập nhật XSA và build platform phải được phân biệt với build application. Hai kết quả có log riêng; báo cáo không dùng một ảnh để chứng minh thay cho cả hai hoạt động.

31. Thông số GPIO do BSP sinh ra

```
77
78 /* Definitions for peripheral AXI_GPIO_0 */
79 #define XPAR_AXI_GPIO_0_COMPATIBLE "xlnx,axi-gpio-2.0"
80 #define XPAR_AXI_GPIO_0_BASEADDR 0x41200000
81 #define XPAR_AXI_GPIO_0_HIGHADDR 0x4120ffff
82 #define XPAR_AXI_GPIO_0_INTERRUPT_PRESENT 0x1
83 #define XPAR_AXI_GPIO_0_IS_DUAL 0x0
84 #define XPAR_AXI_GPIO_0_INTERRUPTS 0x401d
85 #define XPAR_FABRIC_AXI_GPIO_0_INTR 29
86 #define XPAR_AXI_GPIO_0_INTERRUPT_PARENT 0xf8f01000
87 #define XPAR_AXI_GPIO_0_GPIO_WIDTH 0x5
88
89 /* Canonical definitions for peripheral AXI_GPIO_0 */
90 #define XPAR_XGPIO_0_BASEADDR 0x41200000
91 #define XPAR_FABRIC_XGPIO_0_INTR 29
92 #define XPAR_XGPIO_0_HIGHADDR 0x4120ffff
93 #define XPAR_XGPIO_0_COMPATIBLE "xlnx,axi-gpio-2.0"
94 #define XPAR_XGPIO_0_GPIO_WIDTH 0x5
95 #define XPAR_XGPIO_0_INTERRUPT_PRESENT 0x1
96 #define XPAR_XGPIO_0_IS_DUAL 0x0
97 #define XPAR_XGPIO_0_INTERRUPTS 0x401d
98 #define XPAR_XGPIO_0_INTERRUPT_PARENT 0xf8f01000
99
```

Hình 29: Các định nghĩa AXI_GPIO_0 trong xparameters.h

Hình 29 ghi BASEADDR = 0x41200000, HIGHADDR = 0x4120FFFF, GPIO_WIDTH = 0x5 và INTERRUPT_PRESENT = 0x1. Fabric interrupt macro của AXI GPIO là XPAR_FABRIC_AXI_GPIO_0_INTR = 29, phù hợp với GPIO input 5 bit dùng cho buttons.

```
9
0 /* Definitions for peripheral AXI_GPIO_1 */
1 #define XPAR_AXI_GPIO_1_COMPATIBLE "xlnx,axi-gpio-2.0"
2 #define XPAR_AXI_GPIO_1_BASEADDR 0x41210000
3 #define XPAR_AXI_GPIO_1_HIGHADDR 0x4121ffff
4 #define XPAR_AXI_GPIO_1_INTERRUPT_PRESENT 0x0
5 #define XPAR_AXI_GPIO_1_IS_DUAL 0x0
6 #define XPAR_AXI_GPIO_1_GPIO_WIDTH 0x8
7
8 /* Canonical definitions for peripheral AXI_GPIO_1 */
9 #define XPAR_XGPIO_1_BASEADDR 0x41210000
0 #define XPAR_XGPIO_1_HIGHADDR 0x4121ffff
1 #define XPAR_XGPIO_1_COMPATIBLE "xlnx,axi-gpio-2.0"
2 #define XPAR_XGPIO_1_GPIO_WIDTH 0x8
3 #define XPAR_XGPIO_1_INTERRUPT_PRESENT 0x0
4 #define XPAR_XGPIO_1_IS_DUAL 0x0
```

Hình 30: Các định nghĩa AXI_GPIO_1 trong xparameters.h

Hình 30 ghi BASEADDR = 0x41210000, HIGHADDR = 0x4121FFFF, GPIO_WIDTH = 0x8 và INTERRUPT_PRESENT = 0x0. Đây là GPIO LED không có chức năng phát ngắt được bật theo metadata.

Các ảnh BSP/xparameters.h được dùng để đối chiếu GPIO width, interrupt presence, clock frequency, base address và fabric interrupt IDs. Khi build application, tên macro phải bám theo BSP của platform hiện tại.

32. Thông số Timer và GIC trong BSP

```
222
223 /* Definitions for peripheral AXI_TIMER_0 */
224 #define XPAR_AXI_TIMER_0_COMPATIBLE "xlnx,axi-timer-2.0"
225 #define XPAR_AXI_TIMER_0_BASEADDR 0x42800000
226 #define XPAR_AXI_TIMER_0_HIGHADDR 0x4280ffff
227 #define XPAR_AXI_TIMER_0_CLOCK_FREQUENCY 0x5f5e100
228 #define XPAR_AXI_TIMER_0_INTERRUPTS 0x401e
229 #define XPAR_FABRIC_AXI_TIMER_0_INTR 30
230 #define XPAR_AXI_TIMER_0_INTERRUPT_PARENT 0xf8f01000
231
232 /* Canonical definitions for peripheral AXI_TIMER_0 */
233 #define XPAR_XTMRCTR_0_BASEADDR 0x42800000
234 #define XPAR_FABRIC_XTMRCTR_0_INTR 30
235 #define XPAR_XTMRCTR_0_HIGHADDR 0x4280ffff
236 #define XPAR_XTMRCTR_0_COMPATIBLE "xlnx,axi-timer-2.0"
237 #define XPAR_XTMRCTR_0_CLOCK_FREQUENCY 0x5f5e100
238 #define XPAR_XTMRCTR_0_INTERRUPTS 0x401e
239 #define XPAR_XTMRCTR_0_INTERRUPT_PARENT 0xf8f01000
240
```

Hình 31: Các định nghĩa AXI_TIMER_0 trong xparameters.h

Hình 31 ghi BASEADDR = 0x42800000, HIGHADDR = 0x4280FFFF, CLOCK_FREQUENCY = 0x5F5E100 và XPAR_FABRIC_AXI_TIMER_0_INTR = 30. Chuyển đổi 0x5F5E100 cho 100,000,000 Hz. Giá trị này là clock mô tả trong BSP, không phải tần số đo bằng thiết bị ngoài.

```
155
156 #define XPAR_XSCUGIC_NUM_INSTANCES 1
157
158 /* Definitions for peripheral INTC */
159 #define XPAR_INTC_COMPATIBLE "arm,cortex-a9-gic"
160 #define XPAR_INTC_BASEADDR 0xf8f01000
161 #define XPAR_INTC_HIGHADDR 0xf8f01fff
162 #define XPAR_INTC_BASEADDR_1 0xf8f00100
163
164 /* Canonical definitions for peripheral INTC */
165 #define XPAR_XSCUGIC_0_BASEADDR 0xf8f01000
166 #define XPAR_XSCUGIC_0_HIGHADDR 0xf8f01fff
167 #define XPAR_XSCUGIC_0_COMPATIBLE "arm,cortex-a9-gic"
168
```

Hình 32: Các định nghĩa INTC và XSCUGIC trong xparameters.h

Hình 32 ghi vùng INTC có BASEADDR = 0xF8F01000 và HIGHADDR = 0xF8F01FFF; compatible chỉ bộ điều khiển ngắt Cortex-A9. Không tự gọi giá trị này là CpuBaseAddress trong code nếu chưa đối chiếu cấu trúc cầu hình XScuGic của BSP thực tế.

GIC có trách nhiệm quản lý và phân phối ngắt tới CPU, khác với Concat chỉ ghép các tín hiệu ngắt trong PL. Mã cấu hình GIC được giữ nguyên ở mục cấu hình ngoại lệ và GIC để đối chiếu tên API với metadata hiện có.

33. C Source và Hardware Definitions

Các khối code dưới đây giữ nguyên logic của source được cung cấp trong báo cáo: khởi tạo GPIO/Timer, đăng ký ISR, cấu hình GIC và điều khiển LED. Phần phân tích tập trung vào quan hệ giữa software và hardware design.

```
#include "xparameters.h"
#include "xgpio.h"
#include "xtmrctr.h"
#include "xscugic.h"
#include "xil_exception.h"
#include "xil_printf.h"

// Parameter definitions
#define INTC_DEVICE_ID      XPAR_PS7_SCUGIC_0_DEVICE_ID
#define TMR_DEVICE_ID      XPAR_TMRCTR_0_DEVICE_ID
#define BTNS_DEVICE_ID      XPAR_AXI_GPIO_0_DEVICE_ID
#define LEDS_DEVICE_ID      XPAR_AXI_GPIO_1_DEVICE_ID
#define INTC_GPIO_INTERRUPT_ID XPAR_FABRIC_AXI_GPIO_0_INTR
#define INTC_TMR_INTERRUPT_ID XPAR_FABRIC_AXI_TIMER_0_INTR

#define BTN_INT              XGPIO_IR_CH1_MASK
#define TMR_LOAD             0xF8000000

XGpio LEDInst, BTNInst;
XScuGic INTCInst;
XTmrCtr TMRInst;
static int led_data;
static int btn_value;
static int tmr_count;
```

Các header khai báo driver GPIO, Timer, GIC, cơ chế exception và hàm in. LEDInst/BTNInst là hai instance GPIO; TMRInst là Timer; INTCInst là GIC. led_data lưu giá trị ghi ra LED, btn_value lưu kết quả đọc buttons và tmr_count là bộ đếm số lần xử lý Timer theo thuật toán.

TMR_LOAD = 0xF8000000 là Timer reset/load value trong source. BTN_INT sử dụng XGPIO_IR_CH1_MASK. Hai fabric interrupt IDs được lấy từ BSP hiện tại: AXI GPIO = XPAR_FABRIC_AXI_GPIO_0_INTR và AXI Timer = XPAR_FABRIC_AXI_TIMER_0_INTR.

Nhóm định nghĩa	Vai trò
INTC_DEVICE_ID, TMR_DEVICE_ID	Device IDs dùng để khởi tạo GIC và AXI Timer drivers.
BTNS_DEVICE_ID, LEDS_DEVICE_ID	Device IDs ánh xạ hai AXI GPIO instances cho buttons và LEDs.
INTC_GPIO_INTERRUPT_ID, INTC_TMR_INTERRUPT_ID	Fabric interrupt IDs tương ứng AXI GPIO và AXI Timer trong GIC configuration.

34. GPIO ISR

```
void BTN_Intr_Handler(void *InstancePtr)
{
    // Disable GPIO interrupts
    XGpio_InterruptDisable(&BTNInst, BTN_INT);
    // Ignore additional button presses
    if ((XGpio_InterruptGetStatus(&BTNInst) & BTN_INT) !=
        BTN_INT) {
        return;
    }
    btn_value = XGpio_DiscreteRead(&BTNInst, 1);
    // Increment counter based on button value
    // Reset if centre button pressed
    if(btn_value != 1) led_data = led_data + btn_value;
    else led_data = 0;
    XGpio_DiscreteWrite(&LEDInst, 1, led_data);
    (void)XGpio_InterruptClear(&BTNInst, BTN_INT);
    // Enable GPIO interrupts
    XGpio_InterruptEnable(&BTNInst, BTN_INT);
}
```

BTN_Intr_Handler trước hết tạm tắt ngắt kênh GPIO, kiểm tra trạng thái rồi đọc channel 1. Khi btn_value khác 1, code cộng toàn bộ giá trị buttons vào led_data; khi bằng 1, code đặt led_data về 0. Sau đó hàm ghi LED, clear trạng thái ngắt và cho phép lại ngắt GPIO.

Đoạn xử lý	Ý nghĩa theo code
InterruptDisable / GetStatus	Tạm che ngắt và kiểm tra nguồn.
DiscreteRead	Đọc giá trị buttons qua giao diện thanh ghi.
btn_value != 1	Nhánh cộng giá trị; không chỉ tăng cố định một đơn vị.
btn_value == 1	Nhánh reset bộ đếm theo quy ước centre button của bài.
DiscreteWrite / InterruptClear / Enable	Ghi đầu ra, xử lý pending và mở lại ngắt.

Trong control flow hiện tại, nhánh return sớm nằm sau XGpio_InterruptDisable và trước XGpio_InterruptEnable. Nếu nhánh này xảy ra, GPIO interrupt không được re-enable trong chính handler; đây là điểm cần lưu ý khi hoàn thiện ISR.

Nếu nhiều buttons cùng active, code xử lý giá trị số của toàn bộ GPIO bus và không có priority branch riêng cho từng button. Behavioral simulation được diễn giải theo đúng quy tắc này.

35. Timer ISR

```
void TMR_Intr_Handler(void *data)
{
    if (XTmrCtr_IsExpired(&TMRInst,0)){
        // Once timer has expired 3 times, stop, increment counter
        // reset timer and start running again
        if(tmr_count == 3){
            XTmrCtr_Stop(&TMRInst,0);
            tmr_count = 0;
            led_data++;
            XGpio_DiscreteWrite(&LEDInst, 1, led_data);
            XTmrCtr_Reset(&TMRInst,0);
            XTmrCtr_Start(&TMRInst,0);

        }
        else tmr_count++;
    }
}
```

TMR_Intr_Handler kiểm tra XTmrCtr_IsExpired trước khi thay đổi trạng thái. Nếu tmr_count chưa bằng 3, hàm tăng bộ đếm này. Khi vào hàm với tmr_count bằng 3, hàm dừng Timer, đặt tmr_count về 0, tăng led_data, ghi LED, reset và khởi động lại Timer.

Lần xử lý hết hạn	tmr_count trước	tmr_count sau	Tác động lên LED
1	0	1	Không tăng do Timer
2	1	2	Không tăng do Timer
3	2	3	Không tăng do Timer
4	3	0	Tăng led_data một đơn vị

Bảng phân tích trực tiếp nhánh if/else với tmr_count ban đầu bằng 0. Điều kiện được kiểm tra trước phép tăng nên LED được update ở lần Timer expiration thứ tư, sau đó tmr_count trở về 0.

Code đăng ký TMR_Intr_Handler bằng XTmrCtr_SetHandler và kết nối Timer interrupt vào GIC. Khi Timer hết hạn, handler cập nhật tmr_count và sau mỗi bốn lần hết hạn sẽ tăng led_data, ghi LED, reset và restart Timer.

Mốc thời gian trên waveform là thời gian mô phỏng. Không dùng khoảng cách giữa hai xung trên waveform để suy ra chu kỳ thực tế của Timer với TMR_LOAD đang có trong source C.

36. Khởi tạo GPIO, Timer và Main Loop

```
int main (void)
{
    int status;
    //-----
    // INITIALIZE THE PERIPHERALS & SET DIRECTIONS OF GPIO
    //-----
    // Initialise LEDs
    status = XGpio_Initialize(&LEDInst, LEDS_DEVICE_ID);
    if(status != XST_SUCCESS) return XST_FAILURE;
    // Initialise Push Buttons
    status = XGpio_Initialize(&BTNInst, BTNS_DEVICE_ID);
    if(status != XST_SUCCESS) return XST_FAILURE;
    // Set LEDs direction to outputs
    XGpio_SetDataDirection(&LEDInst, 1, 0x00);
    // Set all buttons direction to inputs
    XGpio_SetDataDirection(&BTNInst, 1, 0xFF);

    //-----
    // SETUP THE TIMER
    //-----
    status = XTmrCtr_Initialize(&TMRInst, TMR_DEVICE_ID);
    if(status != XST_SUCCESS) return XST_FAILURE;
    XTmrCtr_SetHandler(&TMRInst, TMR_Intr_Handler, &TMRInst);
    XTmrCtr_SetResetValue(&TMRInst, 0, TMR_LOAD);
    XTmrCtr_SetOptions(&TMRInst, 0, XTC_INT_MODE_OPTION |
XTC_AUTO_RELOAD_OPTION);

    // Initialize interrupt controller
    status = IntcInitFunction(INTC_DEVICE_ID, &TMRInst, &BTNInst);
    if(status != XST_SUCCESS) return XST_FAILURE;

    XTmrCtr_Start(&TMRInst, 0);

    while(1);

    return 0;
}
```

main khởi tạo GPIO LED và buttons, kiểm tra trạng thái trả về và đặt hướng kênh. Sau đó Timer được khởi tạo, đăng ký handler, nạp giá trị reset và bật các tùy chọn interrupt/auto reload. IntcInitFunction được gọi trước XTmrCtr_Start; vòng while(1) không thực hiện polling ngoại vi.

37. Exception và GIC Configuration

```
int InterruptSystemSetup(XScuGic *XScuGicInstancePtr)
{
    XGpio_InterruptEnable(&BTNInst, BTN_INT);
    XGpio_InterruptGlobalEnable(&BTNInst);

    Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
        (Xil_ExceptionHandler)XScuGic_InterruptHandler,
        XScuGicInstancePtr);
    Xil_ExceptionEnable();

    return XST_SUCCESS;
}
```

InterruptSystemSetup bật ngắt tại GPIO và đăng ký XScuGic_InterruptHandler cho exception XIL_EXCEPTION_ID_INT của CPU. Xil_ExceptionEnable cho phép tiếp nhận exception/ngắt theo API. Hàm này là lớp nối giữa cơ chế exception CPU với bộ phân phối handler của GIC.

```
int IntcInitFunction(u16 DeviceId, XTmrCtr *TmrInstancePtr,
                    XGpio *GpioInstancePtr)
{
    XScuGic_Config *IntcConfig;
    int status;

    IntcConfig = XScuGic_LookupConfig(DeviceId);
    status = XScuGic_CfgInitialize(&INTCInst, IntcConfig,
        IntcConfig->CpuBaseAddress);
    if(status != XST_SUCCESS) return XST_FAILURE;

    status = InterruptSystemSetup(&INTCInst);
    if(status != XST_SUCCESS) return XST_FAILURE;

    status = XScuGic_Connect(&INTCInst, INTC_GPIO_INTERRUPT_ID,
        (Xil_ExceptionHandler)BTN_Intr_Handler, (void *)GpioInstancePtr);
    if(status != XST_SUCCESS) return XST_FAILURE;

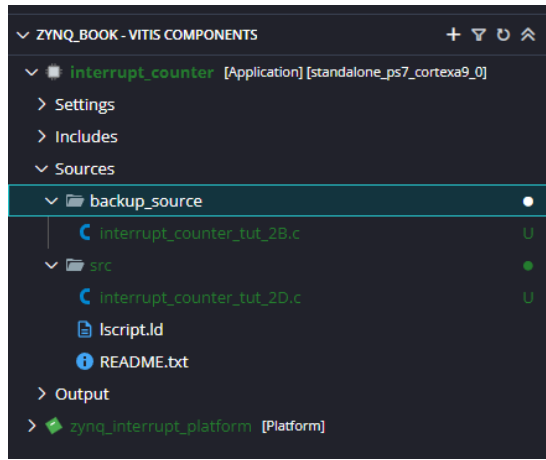
    status = XScuGic_Connect(&INTCInst, INTC_TMR_INTERRUPT_ID,
        (Xil_ExceptionHandler)TMR_Intr_Handler, (void *)TmrInstancePtr);
    if(status != XST_SUCCESS) return XST_FAILURE;

    XGpio_InterruptEnable(GpioInstancePtr, 1);
    XGpio_InterruptGlobalEnable(GpioInstancePtr);
    XScuGic_Enable(&INTCInst, INTC_GPIO_INTERRUPT_ID);
    XScuGic_Enable(&INTCInst, INTC_TMR_INTERRUPT_ID);

    return XST_SUCCESS;
}
```

IntcInitFunction lấy GIC configuration, khởi tạo INTCInst, connect GPIO/Timer interrupt IDs với hai ISR và enable chúng trong GIC. Các status returns được dùng để dừng initialization khi một API call không thành công.

38. Build Application và ELF



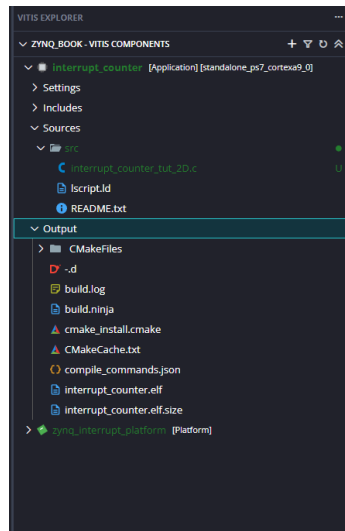
Hình 33: Cây source của application interrupt_counter

Hình 33 thể hiện source interrupt_counter_tut_2D.c trong src và file 2B trong backup_source. Ảnh không đủ để kết luận backup_source nằm ngoài component hoặc chắc chắn bị loại khỏi build; điều này phụ thuộc cấu hình build thực tế.



Hình 34: Log biên dịch source 2D và liên kết ELF

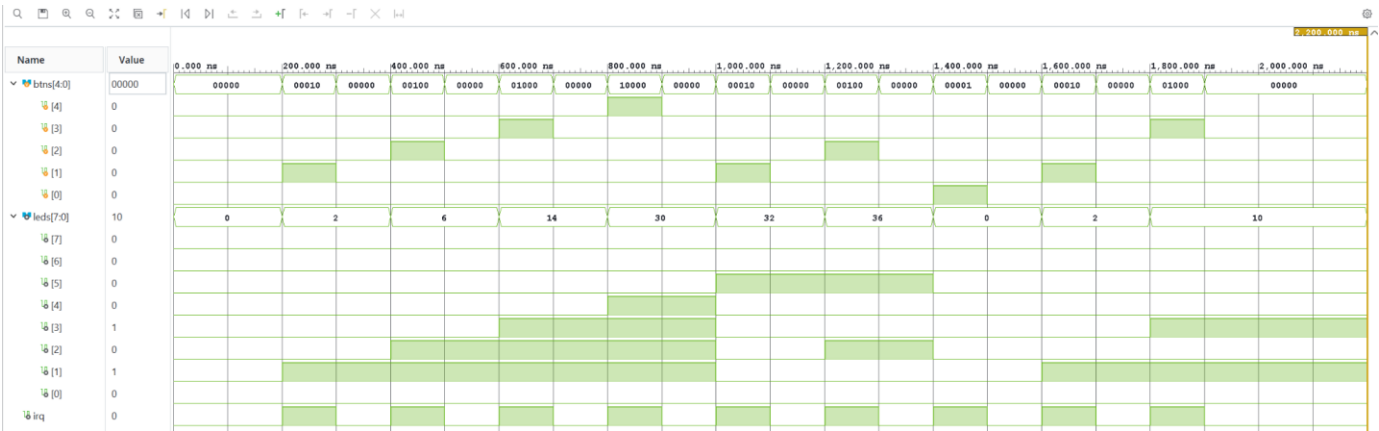
Hình 34 thể hiện compiler tạo interrupt_counter_tut_2D.c.obj và linker tạo interrupt_counter.elf. Đây là build artifact của application được dùng trong phần software flow.



Hình 35: File ELF xuất hiện trong Output của Vitis

Hình 35 cho thấy interrupt_counter.elf xuất hiện trong thư mục Output của application, xác nhận linker đã tạo ELF artifact cho software flow.

39. GPIO Behavioral Simulation



Hình 36: Waveform mô hình GPIO interrupt và LED counter

Hình 36 thể hiện quan hệ giữa stimulus buttons, gpio_irq và LEDs trong mô hình. Theo phần mô tả bài làm, chuỗi buttons có các giá trị 00010, 00100, 01000, 10000 và centre 00001. Các nút tương ứng làm thay đổi bộ đếm; centre đưa LED về 0.

Stimulus	Kỳ vọng theo code GPIO	Phạm vi bằng chứng
00010	Cộng 2 vào led_data	Hành vi mô hình trên waveform.
00100	Cộng 4 vào led_data	Hành vi mô hình trên waveform.
01000	Cộng 8 vào led_data	Hành vi mô hình trên waveform.
10000	Cộng 16 vào led_data	Hành vi mô hình trên waveform.
00001	Đặt led_data về 0	Nhánh centre/reset trong mô hình.

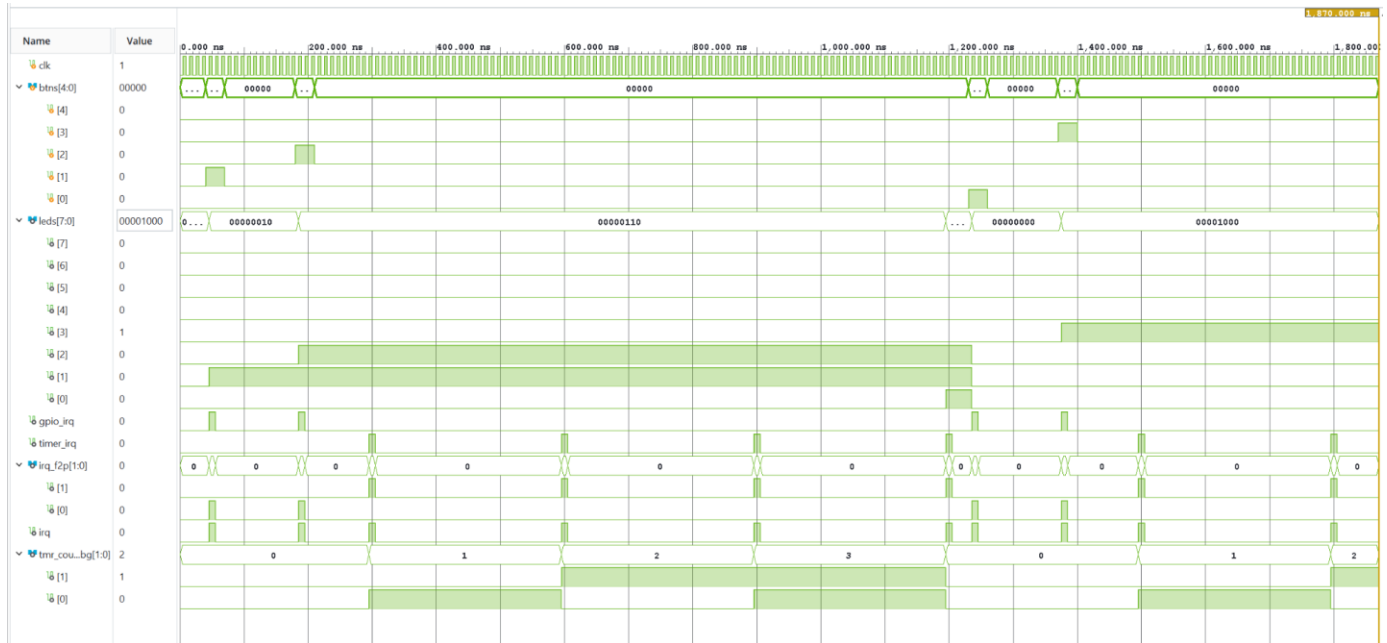
Các giá trị tăng trong bảng được suy ra trực tiếp từ phép cộng btn_value trong GPIO ISR và được kiểm tra trên waveform. Khi đọc một thời điểm cụ thể, cần xét giá trị LED trước sự kiện và chuỗi stimulus trước đó.

40. Phạm vi của Simulation

Behavioral simulation không thực thi ELF trên ARM Cortex-A9 và không phải firmware co-simulation. gpio_irq, led_data và ISR behavior được mô hình hóa để kiểm tra logic mong đợi; vì vậy waveform không đại diện cho software execution timing trên PS.

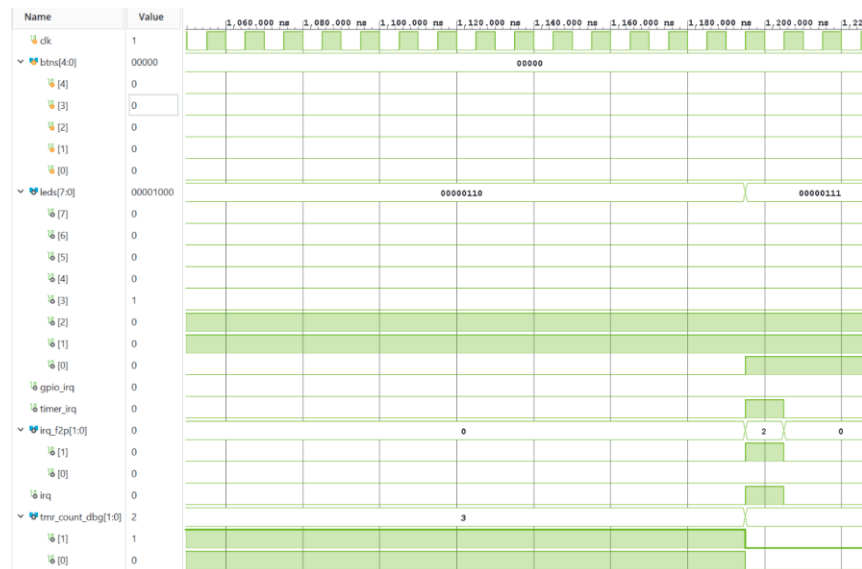
Waveform được sử dụng để kiểm chứng behavioral model theo các stimulus đã trình bày. Kết quả simulation được tách biệt với post-Implementation Timing Report của AXI hardware design.

41. Simulation với hai Interrupt Sources



Hình 37: Waveform tổng thể của GPIO IRQ và Timer IRQ

Hình 37 trình bày behavioral model với hai interrupt sources. `irq_f2p[0]` biểu diễn GPIO interrupt và `irq_f2p[1]` biểu diễn Timer interrupt; cách đánh bit này phù hợp với bus `IRQ_F2P[1:0]` trong Block Design cuối. `tmr_count_dbg` biểu diễn chuỗi $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ của Timer model.

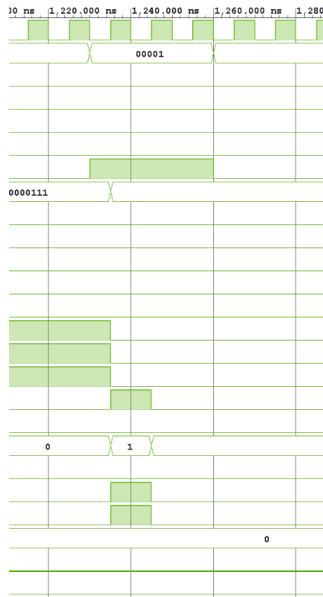


Hình 38: Sự kiện Timer cập nhật LED trong waveform

Hình 38 dùng ảnh phóng to hiện có để quan sát sự kiện Timer: trước khoảng 1.2 μs, LEDs mang giá trị 6 và `tmr_count_dbg` bằng 3; khi Timer IRQ xuất hiện, bộ đếm về 0 và LED tăng lên 7 theo mô tả/giá trị trong ảnh. Không xem mốc thời gian này là độ trễ ngắt trên CPU.

Ảnh đã cắt trước sự kiện GPIO kế tiếp để chỉ trình bày cập nhật do Timer. Hình 39 trình bày riêng khoảng GPIO reset LED. Không dùng một kết quả mô phỏng để kết luận cấu hình GIC của hardware đã được xác nhận.

42. GPIO Reset Event trên LED



Hình 39: Sự kiện GPIO đưa LED về 0 trong waveform

Hình 39 cắt riêng vùng thời gian của sự kiện; các hàng tín hiệu giữ thứ tự như Hình 37, từ trên xuống gồm clock, buttons, LED, GPIO IRQ, Timer IRQ, bus IRQ và bộ đếm. Hình tập trung vào thời điểm btns = 00001, gpio_irq được kích hoạt, irq_f2p biểu diễn nguồn GPIO và LEDs chuyển từ 7 về 0. Đây là kết quả mô hình chức năng của nhánh reset trong BTN_Intr_Handler.

Giá trị bus mô hình	Diễn giải
00	Không có nguồn IRQ được mô hình phát tại thời điểm quan sát.
01	Nguồn GPIO IRQ đang được biểu diễn.
10	Nguồn Timer IRQ đang được biểu diễn.
11	Cả hai bit cùng bằng 1; chưa có kết quả riêng đủ để kết luận xử lý đồng thời.

43. Đánh giá hoàn thành công việc và nhận xét

44. Đối chiếu yêu cầu và kết quả

Yêu cầu	Trạng thái	Bằng chứng và giới hạn
Mô tả PS/PL và các IP	Đạt	Hình 1–10; bảng cổng.
Tích hợp GPIO/Timer/Concat	Đạt	Hình 12 và 15; Concat dout[1:0] nối IRQ_F2P[1:0].
Validate Design	Đạt	Hình 16, thông báo thành công.
Synthesis	Đạt	Hình 18, synth_design Complete!.
Implementation/bitstream	Đạt	Hình 19; write_bitstream Complete!, Failed Routes = 0. TIMING-18 warnings được phân tích ở phần Timing.
Schematic	Đạt	Hình 20; Schematic top-level của wrapper.
Utilization	Đạt	Hình 21 và report_utilization; có LUT, Register, BRAM, DSP, I/O và Clocking.
Timing đầy đủ	Đạt	Hình 22; WNS = 3.522 ns, WHS = 0.106 ns, WPWS = 4.020 ns; 0 failing endpoint.
Power đầy đủ	Đạt	Hình 23; Total On-Chip Power = 1.684 W, Confidence level = Low.
Bản giao/cập nhật XSA	Đạt	Hình 24–27.
Platform và application	Đạt trong phạm vi build	Các hình build platform/application và ELF artifact.
Phân tích mã C	Đạt	Phân tích GPIO ISR, Timer ISR, GIC, main loop và BSP parameters.
Mô phỏng hành vi	Đạt	Các waveform kiểm chứng GPIO event, Timer event và irq_f2p[1:0].

45. Resource, Timing và Power Summary

Resource Utilization cho thấy phần PL sử dụng 1.61% Slice LUTs và 0.86% Slice Registers, không dùng Block RAM Tile hoặc DSP. Timing Summary đạt WNS = 3.522 ns, WHS = 0.106 ns và WPWS = 4.020 ns tại clock 100 MHz, không có failing endpoint. Power Report ước lượng Total On-Chip Power = 1.684 W với Confidence level = Low.

Resource Utilization, Timing, Power, software build và behavioral simulation kiểm chứng các khía cạnh khác nhau của hệ thống. Các report này được trình bày độc lập để tránh dùng một loại kết quả thay cho một loại kiểm chứng khác.

46. Kết luận

Thiết kế hoàn thiện chuỗi HW/SW gồm ZYNQ7 Processing System, hai AXI GPIO, AXI Timer, AXI SmartConnect, Processor System Reset và Concat. Block Design cuối xác nhận interrupt bus 2 bit từ GPIO/Timer tới IRQ_F2P[1:0]; Address Editor xác nhận ba vùng địa chỉ AXI; Synthesis, Implementation, Resource Utilization, Timing và Power reports đều đã được đối chiếu.

Kết quả Timing đáp ứng các timing constraints đã khai báo ở 100 MHz, đồng thời Methodology Report vẫn ghi TIMING-18 đối với I/O delay constraints. Power Report có Confidence level = Low nên được hiểu là estimate từ implemented netlist. Behavioral simulation xác nhận logic xử lý hai interrupt sources trong phạm vi mô hình của bài Lab.

47. Hình nhóm

